

DocuMate

Name:

Zaid Patel - 24MCA-40

Amir Khan – 24MCA-21

1. Project Overview

Product Name: DocuMate - Your Code's Story, Automatically Told

Type: Standalone Cross-platform Desktop Developer Tool

Purpose: Intelligent project documentation and configuration file automation that triggers on Git operations, providing template-based file generation and seamless Git integration through a unified desktop interface.

Key Features

- Triggered by Git push operations with automatic file detection
- Template-based project setup (9 categories including AI/ML, Web Dev, DevOps, etc.)
- Interactive side-by-side diff viewer for README and configuration files
- One-click Git operations (add, commit, push) for all file updates
- Local file-based storage with embedded database
- Automated generation of Dockerfile, .env.example, and .gitignore

2. System Architecture

The system is composed of the following components:

- **Spring Boot Backend:** Manages business logic, Git operations, and REST API endpoints
- **React Native Frontend (Electron):** Provides cross-platform desktop UI with diff viewer and editor
- **Embedded H2 Database:** File-based SQL database for local data storage
- **Git Integration Service:** Monitors Git activity and performs operations using JGit
- **Template Engine:** Generates intelligent file content based on project categories
- **File Watcher Service:** Monitors Git repositories for push events and changes

3. Core Components

3.1 Installation & Setup

- Single executable JAR file for all platforms (Windows, macOS, Linux)
- Zero-configuration setup - download and run with Java 17+
- First-launch project registration wizard
- Automatic Git repository validation and hook installation

- Local data storage in user home directory (~/.documate/)
- Template selection from 9 predefined project categories

3.2 Trigger System

- **Git Hook Integration:** Auto-installs post-push hooks in registered repositories
- **Background File Watcher:** Monitors .git folders using Java WatchService
- **Template Router:** Directs projects to appropriate category-based generators
- **Event Processor:** Handles Git push detection and triggers file generation workflows
- **Diff Analyzer:** Compares current and proposed file changes intelligently

3.3 User Interface

Component	Description
Home Dashboard	Grid view of all projects with +New project button
Project Registration	Form for title, Git repo, local path with template selection
Template Selector	9-category template chooser (AI/ML to EdTech)
Diff Viewer	Side-by-side file comparison with editing capability
File Generator	Creates Dockerfile, .env.example, .gitignore based on template
Push Controller	One-click Git add/commit/push with appropriate messages

3.4 Database Layer

- **Database:** Embedded H2 file-based SQL database
- **Storage Location:** ~/.documate/data/ directory
- **Key Collections:**
 - **projects:** Project metadata, paths, and template associations
 - **templates:** Predefined project categories and generation rules
 - **file_versions:** Complete history of all generated file versions
 - **git_operations:** Audit log of all automated Git commands

4. Workflow Sequence

4.1 Detection

- User launches application and clicks "+New" on home dashboard
- Enters project details: Title, Git Repo Link, Local Path
- Selects from 9 template categories for intelligent file generation

- System validates Git repository and installs necessary hooks
- Initial project scan creates baseline understanding in local database

4.2 Git Push Trigger & README Update

- User executes git push in terminal/IDE
- Git hook triggers DocuMate application
- System generates updated README based on changes and project template
- Diff screen displays: Old README ↔ New README (editable)
- User reviews/edit → clicks "Push" button
- Auto-executes: git add ., git commit -m "readme updated", git push -u origin main
- Returns to home dashboard upon completion

4.3 Configuration File Generation

- User selects project from home dashboard
- Views latest README (read-only preview)
- Clicks file generation options:
 - Create/Edit Dockerfile
 - Create/Edit .env.example
 - Create/Edit .gitignore
- System generates intelligent file content based on project template
- Diff screen shows: Old File ↔ New File (editable)
- User clicks "Push" → Auto Git operations with contextual commit messages
- Returns to project view after successful push

4.3 Template-Based Intelligence

1. Artificial Intelligence & Machine Learning Projects
2. Data Science & Analytics Projects
3. Web & Mobile Development Projects
4. Cloud Computing & DevOps Projects
5. Cybersecurity & Network Security Projects
6. Software Management System Projects
7. Blockchain & FinTech Projects
8. Game Development Projects
9. Educational / EdTech Projects

5. Technology Stack Summary

Component	Technology	Purpose/Justification
Application Framework	Spring Boot	Standalone executable with embedded Tomcat
User Interface	React	Modern desktop UI with native performance
Database	H2 Database	Embedded file-based SQL, zero configuration
Git Integration	JGit	Pure Java Git, no external dependencies
Template Engine	Thymeleaf	Intelligent file generation from templates
File Processing	Java NIO	Efficient filesystem operations and watching
Packaging	Maven Shade Plugin	Single executable JAR with all dependencies
Markdown Handling	CommonMark	Standard-compliant Markdown processing

6. Security & Permissions

Concern	Design Approach
File Access	Restricted to registered project directories only
Git Operations	Safe operations only, no force pushes or rebases
Local Storage	All data in user's home directory, complete privacy
Network Access	Optional, only for Git remote operations when needed
User Approval	No file changes without explicit user confirmation
System Permissions	Respects and works within user's file permissions

7. User Scenarios

7.1 First-Time Setup:

- User downloads documate.jar from project repository
- Double-clicks to launch (requires Java 17+ runtime)
- Views home dashboard and clicks "+New" project
- Registers project with local Git repository path
- Selects appropriate template category from 9 options
- System performs initial scan and installs Git hooks
- Ready for automated documentation updates

7.2 Daily Development & Documentation Workflow

- User develops code and makes local commits
- Executes git push command
- DocuMate automatically triggers and shows README diff
- User reviews changes, edits if needed, clicks "Push"
- System handles complete Git workflow automatically
- Returns to dashboard, user continues development
- For new configurations: user accesses project → generates files → one-click push

8. Future Enhancements

- Expanded template library with community contributions
- Team collaboration features for shared project contexts
- Advanced Git workflow customization options
- Cloud synchronization backup (optional)
- Plugin system for custom file generators
- Integration with CI/CD pipelines and project management tools

9. Why This Stack Is Suitable

Tech	Stack Benefit
Spring Boot + Electron	True standalone application, no web server required
Embedded H2 Database	Zero configuration, file-based, completely portable
JGit Integration	No external Git installation needed, pure Java solution
Single JAR Packaging	Download and execute simplicity across all platforms
Local-First Design	Complete data privacy, works entirely offline
Template Architecture	Scalable project type support with consistent quality

10. Summary

DocuMate is a completely self-contained desktop application that revolutionizes project documentation through intelligent automation and seamless Git integration. The unified Spring Boot and Electron architecture provides a robust, cross-platform solution that requires only a Java runtime to operate. With its template-driven approach spanning 9 project categories and embedded local database storage, DocuMate offers a zero-configuration experience that maintains comprehensive project documentation and configuration files with minimal developer intervention. The application enhances developer productivity by automating repetitive documentation tasks while preserving full control through intuitive visual interfaces and explicit approval workflows.